UNIVERSIDAD CATÓLICA ARGENTINA
SANTA MARÍA DE LOS BUENOS AIRES

Facultad de Ingeniería y Ciencias Agrarias

Carrera: Ingeniería Informática

Cátedra: Protocolos de Internet

PRÁCTICA 1 – Cliente Servidor

Fecha: 13 / 04 / 2026

Estudiante/s: Guillermo Alberto Rizzuto, Agustín Martín Safronchik, Gonzalo Val

Legajo/s: 45129620, 44517212, 151722804

Ejercicios a realizar durante la clase.

B-1) Comparar cliente con select y bloqueante.

B-2) Modificar el cliente bloqueante para que sea no bloqueante.

B-3) Compara cliente con select y no bloqueante sin select.

B-4) Conclusiones.

Consigna del TP 1

1) Explicar qué es un socket y los diferentes tipos de sockets.
2) Cuáles son las estructuras necesarias para operar con sockets en el modelo C-S y cómo se hace para ingresar los datos requeridos. -- Explicar con un ejemplo.
3) Explicar modo bloqueante y no bloqueante en sockets y cuáles son los "sockets calls" a los cuales se pueden aplicar estos modos. -- Explicar las funciones necesarias.
4) Describir el modelo C-S aplicado a un servidor con Concurrencia Real.
5) Escribir un ejemplo en lenguaje C distinto del ya presentado.
6) Cómo se cierra una conexión C-S ?. Métodos que eviten la pérdida de informacion.
7) Probar el código del chat entre cliente y servidor. Cambiar tipo de socket y volver a probar.
8) Presentar las respuestas en archivos Jupyter Notebook o PDF

Soluciones

Nota sobre el entorno de ejecución: Para el desarrollo de estas experiencias se utilizó macOS (Intel) como sistema base. Al ser un sistema operativo de arquitectura Unix nativa, no utilizamos capas de compatibilidad adicionales como WSL.

B-1) La diferencia principal es cómo esperan los datos. El cliente bloqueante detiene la ejecución del programa en **recv()** hasta que llegan datos. Un cliente con select() permite monitorear varios descriptores (como el teclado y el socket) al mismo tiempo, actuando solo cuando hay actividad.

B-2) Para que un socket no sea bloqueante, usamos **fcntl** para setear el flag **O_NONBLOCK**. Esto hace que **recv()** devuelva un error si no hay datos, en lugar de esperar. Se aplicó la misma lógica al descriptor **STDIN_FILENO**. En modo no bloqueante, un retorno de **-1** no siempre significa un error de red. Se incorporó una lógica de control para verificar si **errno** es igual a **EWOULDBLOCK** o **EAGAIN**, lo cual simplemente indica que el recurso no tiene datos listos en ese instante.

B-3) En el caso del cliente no bloqueante sin select, el programa entra en un bucle infinito preguntando si llegaron los datos constantemente. Esto consume mucha CPU a menos que se use **sleep()**, pero esto quita precisión. Por otro lado, con select, el programa se "duerme" y es el kernel del sistema operativo quien lo despierta sólo cuando hay algo que leer. Es mucho más eficiente en recursos.

B-4) Conclusiones

Bloqueante: Es el más simple de programar pero ineficiente para interfaces interactivas, ya que el programa se detiene totalmente en la lectura.

No Bloqueante: Es útil si el programa tiene que hacer cálculos pesados mientras espera, pero el polling (preguntar constantemente) es costoso para el CPU.

Select: Es la opción óptima para manejar múltiples entradas (teclado + red) de forma eficiente, permitiendo que el sistema operativo gestione la espera por nosotros.

1) **Explicar qué es un socket y los diferentes tipos de sockets.**

Un socket, enchufe, es un método para la comunicación entre un programa del cliente y un programa del servidor en una red. Se define como el punto final en una conexión. Los sockets se crean y se utilizan con un sistema de peticiones o de llamadas de función a veces llamados interfaz de programación de aplicación de sockets.

Para las comunicaciones en redes la llamada al sistema que se usa es el sokcet(). Devuelve un descriptor de comunicaciones para usar las llamadas al sistema especializadas tales como: send(), recv(), read() o write().

Los sockets existentes son:

- Raw sockets (sockets puros) "SOCK_RAW": son los sockets que se utilizan cuando queremos acceder directamente a la capa de red y a la capa de enlace de datos.

  Incluye encabezados IP, protocolos y datos subsecuentes, con lo que se puede enviar cualquier tipo de paquete IP por este socket. Asimismo se pueden recibir cualquier tipo de paquetes que lleguen al host si se hace un read() desde el.
- Stream socket (sockets de flujo) "SOCK_STREAM": definen flujos de comunicación en dos direcciones, confiables y con conexión TCP, e IP para el enrutamiento.

  La información llega de manera secuencial y sin errores, es decir, si se envían los mensajes en el orden "1, 2" llegaran al otro extremo en el orden "1, 2" y sin errores.
- Datagram sockets (sockets de datagramas) "SOCK_DGRAM": Utilizan UDP, no tienen que mantener una conexión abierta, se usan generalmente para transferencia de información por paquetes.

  Si se envía un datagrama puede que llegue o no, llegue fuera de secuencia y, si llega, los datos que contiene el paquete no tendrán errores. Simplemente se arma un paquete, se coloca una cabecera IP con la información de destino y se envía.

  Por todas estas razones, si se quiere asegurar que los paquetes lleguen a destino se debe de implementar un protocolo por arriba de UDP.

2) **Cuáles son las estructuras necesarias para operar con sockets en el modelo C-S y cómo se hace para ingresar los datos requeridos. -- Explicar con un ejemplo.**

El modelo cliente servidor es una solución que se da al problema de recuento. Este modelo establece que en, todo par de aplicaciones que se comuniquen, la aplicación de un lado debe iniciar la ejecución y esperar, indefinidamente, a que llegue una solución del otro lado.

El extremo que ejecuta la comunicación se la conoce como cliente y el que espera se la conoce como servidor.

Las estructuras necesarias para operar con socket en el modelo Cliente-Servidor serán las siguientes:

- sockaddr: esta estructura genérica mantiene información de direcciones de socket para diversos tipos de sockets. Se reservan 2 bytes para especificar el nombre de la familia y un array de 14 bytes para almacenar la dirección.

  Su representación será:

  ```
  struct sockaddr{
          unsigned short sa_family;
          char     sa_data[14];
  };
  ```

  Sa_family admite varios valores y sa_data contiene una dirección y un numero de puerto de destino para el socket, no es adecuado empaquetar a mano una dirección y un numero de puerto dentro.

- sockaddr_in: no todas las familias se adaptan a sockaddr. Toda familia tiene especifica su estructura que define la dirección destino, en el caso de TCP/IP se tienen 2 bytes para identifica la familia, 2 bytes para indicar el numero de puerto, 4 bytes para la dirección IP y 8 bytes no utilizados.

  ```
  struct sockaddr_in{
          short int          sin_family;
          unsigned short int sin_port;
          struct in:addr   sin_addr;
          unsigned char  sin_zero[8];
  };
  ```

sin_family guarda la familia de dirección, sin_port, el numero de puerto, sin_addr la dirección de internet y sin_zero se agrega para que la nueva estructura tenga el mismo tamaño que el struct sockaddr.

- in_addr: es una estructura anidada dentro de sockaddr_in la cual guarda la dirección IP. Su estructura será

```
struct in_addr {
        unsigned long s_addr;
};
```

3) **Explicar modo bloqueante y no bloqueante en sockets y cuáles son los "sockets calls" a los cuales se pueden aplicar estos modos. -- Explicar las funciones necesarias.**

Un socket en modo bloqueante hace que el programa se detenga, espera, hasta que una operación se complete. En cambio, en un socket en modo no bloqueante las funciones del mismo no se detienen.

Por default los sockets son bloqueante. Significa que por defecto cuando ejecutamos alguna función de socket que no puede ser concreta en el momento el proceso se pone a dormir.

Las "sockets calls" a las cuales se le aplican estos modos se las pueden clasificar en cuatro categorías:

- Entrada: Estas funciones son utilizadas, ya sea por el servidor como por el cliente, para recibir datos a través de una conexión. Si se llaman a las funciones read, readv, recv, recvfrom y recvmsg y no hay datos que lleguen, en caso de que el puerto sea bloqueante, el proceso se dormirá. En cambio, si el socket es no bloqueante si no llegan datos las funciones retornaran el error EWOULDBLOCK.
- Salida: Sean las funciones utilizadas para enviar datos a través de una conexión, ejemplos de estas funciones serian write, writev, send, sendto o sendmsg.

  En el caso de que el socket sea bloqueante si el buffer por el que se envían los datos esta lleno, el proceso se bloque hasta que haya lugar. Por su parte si el puerto en no bloqueante y no hay espacio en el buffer por el que se envían los datos las funciones retornaran EWOULDBLOCK.
- Aceptando conexiones entrantes: si se llama la función, en una socket bloqueante, accept() y no hay una conexión entrante se bloquea el

proceso. En el caso de ser un socket no bloqueante se arrojará EWOULDBLOCK.

- Iniciando conexiones: Para TCP si se llama a connect() esta función se bloquearía ya que se realiza el hadshake de 3 vias y el proceso de bloquearía mínimamente por un tiempo de RTT.

Las funciones indicadas, en las previas categorías, son las siguientes:

- read(): usado tanto por el cliente como el servidor para recibir datos a través de una conexión TCP.Si no se leen datos ese bloqueara el proceso.
- recv(): recibe datos a un socket conectado, devolviendo la cantidad de bytes recibidos.
- recvFrom(): recibe datos a un socket no conectado, devolviendo la cantidad de bytes que recibio.
- write(): usado tanto por el cliente como por el servidor para enviar datos a través de una conexión TCP.bloque la ejecución hasta que pueda enviar datos por la conexión TCP y hacer espacio libre para los datos restantes.
- send(): envía datos a un socket determinado devolviendo la cantidad de bytas que se envió.
- sendTo(): envia datos a un socket no conectado, devolviendo la cantidad de bytes enviados.
- accept() : es usado del lado del servidor para aceptar una conexión entrante e intentar crear una nueva conexión TCP con el cliente remoto.
- connect(): es una función utilizada del lado del cliente para asignar un numero de puerto al socket. EN el caso de TCP intentara establecer una nueva conexión TCP con un socket a la escucha de otro puerto y dirección IP.

4) **Describir el modelo C-S aplicado a un servidor con Concurrencia Real.**

Nuevamente, definiremos al modelo Cliente-Servidor como una solución propuesta al problema del reencuentro. En este modelo un extremo ejecuta la comunicación, el “cliente”, y debe esperar indefinidamente la respuesta del otro extremo, conocido como “servidor”.

Definiremos a la concurrencia como la computación simultanea de dos o mas procesos. Es decir, los procesos son concurrentes cuando se ejecutan al mismo tiempo independientemente entre ellos.

Podremos diferenciar tres tipos diferentes de servidores “Iterativo”, “Concurrente” y “Concurrencia Aparente”. Definiéremos como concurrencia real cuando un servidor es capaz de atender a varios clientes a la vez con diferentes procesos.

Por su parte la “Concurrencia Aparente” ocurrirá cuando un servidor utiliza un único proceso para atender a varios clientes a la vez.

El modelo cliente servidor para la concurrencia real funcionara de la siguiente manera:

Cuando el servidor recibe y acepta la conexión del cliente, bifurca una copia de si mismo y permite al hijo manejar al cliente. El servidor fork() una copia de si mismo y permite que el hijo maneje a este cliente desde un puerto n, pero al mismo tiempo, sigue escuchando desde el puerto n también.

Cuando otro cliente solicita conectarse a este servidor. Este servidor nuevamente hace fork() para otra copia de si mismo y deja que este segundo hijo maneje a ese cliente mientras que simultáneamente mantiene escucha desde el puerto n y sigue sirviendo al primer cliente por el primer puerto asignado.

Aunque ambos sockets usan el mismo puerto local en la maquina del servidor, se indican mediante descriptores de archivo de socket distintos, la llamada del servidor devuelta de funciones socket() y accept().

Por su parte el comando fork() crea un nuevo proceso separado para cada cliente dividiendo el proceso actual en dos procesos: un padre y un hijo. El nuevo proceso es una copia del proceso padre.

Cuando el hijo salga enviara una señal al padre para informarle que ha finalizado la ejecución. El sistema operativo no eliminara por completo este proceso cuando salga, el padre debe limpiar toda la información de sus hijos, si no lo hacen estos hijos quedan y no hacen nada pues están terminados.

Un padre realiza la limpieza después de un proceso hijo usando la familia de comando wait().

Cada hijo atiende con un socket distinto pero mismo puerto.

5) **Escribir un ejemplo en lenguaje C distinto del ya presentado.**

En este ejemplo utilizamos UDP con SOCK_DGRAM que puede ser mas simple ya que no requiere establecer una conexión previa con listen() ni accept(), pero tiene sus contras.

Servidor udp:

```
echo '#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define MAX 1024
#define PORT 9000

int main() {
    int sockfd;
    struct sockaddr_in server_addr, client_addr;
    char buffer[MAX];

    // Crear socket UDP (SOCK_DGRAM)
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd == -1) {
        perror("Error al crear el socket UDP");
        exit(1);
    }

    // Configurar estructura
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    // Asociar dirección y puerto (Bind)
    if (bind(sockfd, (const struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
        perror("Bind falló");
        exit(1);
    }

    printf("Servidor UDP esperando datagramas en el puerto %d...\\n", PORT);

    socklen_t addr_size = sizeof(client_addr);
    while (1) {
        memset(buffer, 0, MAX);

        // Recibir datagrama (no hay accept en UDP)
        int bytes = recvfrom(sockfd, buffer, MAX, 0, (struct sockaddr *)&client_addr, &addr_size);
        if (bytes > 0) {
            printf("Mensaje recibido de %s: %s", inet_ntoa(client_addr.sin_addr), buffer);

            // Responder al cliente (Eco)
            sendto(sockfd, buffer, bytes, 0, (struct sockaddr *)&client_addr, addr_size);
        }
    }

    close(sockfd);
    return 0;
}' > servidor_udp.c
```

Cliente udp:

```
echo '#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define MAX 1024
#define PORT 9000

int main() {
    int sockfd;
    struct sockaddr_in server_addr;
    char buffer[MAX];

    // Crear socket UDP
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd == -1) {
        perror("Error al crear el socket");
        exit(1);
    }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);

    printf("Cliente UDP listo. Escriba un mensaje:\\n");

    while (1) {
        printf("> ");
        fgets(buffer, MAX, stdin);

        // Enviar al servidor sin necesidad de connect()
        sendto(sockfd, buffer, strlen(buffer), 0, (struct sockaddr *)&server_addr, sizeof(server_addr));

        if (strncmp(buffer, "FIN", 3) == 0) break;

        // Recibir el Eco del servidor
        memset(buffer, 0, MAX);
        socklen_t addr_size = sizeof(server_addr);
        recvfrom(sockfd, buffer, MAX, 0, (struct sockaddr *)&server_addr, &addr_size);
        printf("Respuesta del servidor: %s", buffer);
    }

    close(sockfd);
    return 0;
}' > cliente_udp.c

gcc cliente_udp.c -o cliente_udp
```

6) **Cómo se cierra una conexión C-S ?. Métodos que eviten la pérdida de informacion.**

Metodos de cierre:

- Cierre por aplicación: En el código se usa la palabra clave “FIN”. Cuando el sistema detecta esta cadena, sale del bucle principal y ejecuta la función de cierre.
- Función close(): Se utiliza para liberar el descriptor del socket tanto en el cliente como en el servidor.

- Función shutdown(): Es más avanzado que close(). Permite cerrar la comunicación en una sola dirección (ej. dejar de enviar pero seguir escuchando). Esto evita la pérdida de información si el servidor todavía tiene datos en el buffer para enviarle al cliente antes de terminar definitivamente.

7) **Probar el código del chat entre cliente y servidor. Cambiar tipo de socket y volver a probar.**

   Al cambiar a SOCK_DGRAM (UDP) ya no existe una conexión permanente y el servidor no necesita hacer un accept para recibir datos. Tiene como ventaja que la comunicación es mas rapida, pero si se pierde un paquete (por ejemplo por una falla de red), el receptor nunca lo volverá a pedir y el programa seguirá como si nada.

   Lo que hicimos fue cambiar las llamadas a las funciones send() y recv() por sendto() y recvfrom().